DSA LAB - Programs Explained from Scratch

For every program you get: the program name, the idea in simple words, then the code **cut into small pieces**, each piece followed by a line-by-line explanation. Read Part 0 first, because it explains the basic syntax used everywhere.

No.	Program	Language
1	Student Database with Linear Search	C++
2	Sort Student IDs and Binary Search	C++
3	Bubble Sort of Students by CGPA	C++
4	Binary Search with Recursion on Student IDs	C++
5	Stack using Linked List	C++
6	Infix to Postfix using Array Stack	C++
7	Infix to Prefix using Array Stack	C++
8	Circular Queue using Array of Size 5	C++
9	BST: Create, Traversals and Duplicate Handling	Python
10	BST: Insertion and Search	Python
11	BST: Deletion (leaf, one child, two children)	Python
12	BST: Depth and Leaf Nodes	Python
13	Kruskal's Algorithm with Adjacency List	Python

No.	Program	Language
14	Prim's Algorithm with Adjacency Matrix	Python
15	Heap Sort	Python
16	Merge Sort for Delivery Orders	Python
17	Fractional Knapsack	Python
18	Naive String Matching	Python
19	BFS Shortest Path in a Maze	Python
20	A* Search in a Maze	Python

Part 0A: C++ basics (for Programs 1-8)

- `#include <iostream>` → brings in ready-made tools. `iostream` gives `cout` (print) and `cin` (read).
- `using namespace std;` → lets us write `cout` instead of `std::cout`.
- `int main() { ... }` → every program starts here. Code inside `{ }` is one block.
- Every statement ends with a semicolon `;`.
- **Variables** have a type: `int` (whole number), `float` (decimal number), `char` (one character), `string` (text). Example: `int n;`
- `cout << "Hello" << x << endl;` prints text and values. `endl` (or `\n`) = new line.
- `cin >> n;` waits for the user to type a value and stores it in `n`.
- `=` stores a value (`x = 5`). `==` compares (is equal?). Others: `!=` not equal, `<`, `>`, `<=`, `>=`.
- `&&` means AND, `||` means OR, `!` means NOT. `i++` adds 1 to `i`. `x += 3` means `x = x + 3`.
- **Array**: a row of boxes of the same type: `int a[5];` gives `a[0]` to `a[4]` (counting starts at 0).
- `if (...) { ... } else { ... }` → do something only when the condition is true.
- `for (int i = 0; i < n; i++) { ... }` → repeat n times. `while (...) { ... }` repeats while a condition is true. `do { ... } while (...)` runs at least once. `break;` leaves a loop.
- **Function**: a named block of code: `int add(int a, int b) { return a + b; }`. `void` means it returns nothing.
- **struct**: our own box with several parts, accessed with a dot: `s.id`.

- **Pointer**: a variable that stores the *address* of something. `new` reserves memory, `delete` gives it back. `p->data` means "the `data` part of what `p` points to".
- `//` starts a comment (ignored by the computer).

Part 0B: Python basics (for Programs 9-20)

1. **Python runs top to bottom**, one line at a time. There is no `main()`, no `;` at the end of lines, and no `{ }` braces.
2. **Indentation (spaces at the start of a line) is important.** A line ending with a colon `:` means "the indented lines below belong to me" (inside an `if`, `for`, `def`, `class`...). Never mix up the spaces.
3. `# text` is a comment (ignored by Python).
4. **Variables** need no type: `x = 5`, `name = "Ravi"`. A single `=` stores a value. `==` compares two values (is equal?). Others: `!=`, `<`, `>`, `<=`, `>=`.
5. **print**: `print("Hello", x)` shows values separated by a space and goes to the next line. `print(x, end=" ")` stays on the same line. `print()` just starts a new line. Inside text, `\n` means new line.
6. **input**: `input("Enter: ")` shows the message and returns what the user typed **as text**. `int("5")` turns text into the whole number 5; `float("2.5")` turns it into a decimal number.
7. **List** = ordered collection: `a = [10, 20, 30]`. Counting starts at 0: `a[0]` is 10. `len(a)` is 3. `a.append(40)` adds at the end. `a[:2]` = first two items, `a[2:]` = from index 2 to the end. `a.sort()` sorts it. `a.reverse()` reverses it. `a.extend(b)` adds all items of `b`.
8. **Reading many numbers in one line:** `list(map(int, input().split()))`. `.split()` cuts the typed text at spaces into pieces, `map(int, ...)` converts each piece to a number, `list(...)` makes a list.
9. **Tuple** = a fixed group like `(2, 3)` (row, column). `t[0]` is the first item.
10. **Dictionary** = pairs `key: value`: `d = {"Gate": 5}`; `d["Gate"]` gives 5. `x in d` / `x not in d` checks if a key exists.
11. **if / elif / else**: runs the first block whose condition is true. Combine conditions with `and`, `or`, `not`. `x is None` checks that x is "nothing" (`None` is like NULL).
12. **Loops**: `for i in range(5):` gives i = 0,1,2,3,4. `range(2, 5)` gives 2,3,4. `range(5, 0, -1)` counts down 5,4,3,2,1. `for item in a:` takes each item of a list. `while condition:` repeats while true. `break` leaves a loop.
13. **Functions**: `def add(a, b):` defines one; `return a + b` gives a value back; `add(2, 3)` calls it. A function can call itself (**recursion**).
14. **Class**: a blueprint for objects (we use it for tree nodes). `__init__` runs when an object is created; `self` means "this object".
15. **Handy things:** `a, b = b, a` swaps two values. `for r, c in pairs:` unpacks each pair. `//` = whole-number division (7 // 2 is 3). `abs(x)` = positive value, `max(a, b)` = bigger. `round(x, 2)` = 2 decimals. `0 <= x < 5` means x is between 0 and 4. `+=` adds and stores (`i += 1`).

16. **import**: brings in extra ready-made tools, like `from collections import deque`.

PART A: C++

Program 1: Student Database with Linear Search (C++)

Idea in simple words: We store many students (ID, name, CGPA) in memory that we reserve while the program runs. Then we look for one student by checking every record one by one from the start until the ID matches. That one-by-one checking is called **Linear Search**.

```
C/C++
#include <iostream>
#include <string>
using namespace std;
```

- `#include <iostream>` → brings in the tools for printing (`cout`) and reading (`cin`).
- `#include <string>` → brings in the tool for storing text (`string`), used for names.
- `using namespace std;` → lets us write `cout` instead of `std::cout`.

```
C/C++
struct Student {
    int id;
    string name;
    float cgpa;
};
```

A `struct` is our own custom "box" that groups related data together.

- `int id;` → a whole number for the student ID.
- `string name;` → text for the name.
- `float cgpa;` → a decimal number for the CGPA.
- `};` → end of the struct (the semicolon is required). Now `Student` is a new type, like `int`, but with 3 parts inside.

```
C/C++
int main() {
    int n;
```

```
    cout << "Enter number of students: ";
    cin >> n;
```

- `int main() {` → every C++ program starts running here.
- `int n;` → makes a variable `n` (a whole number) to store how many students.
- `cout << "..."` → prints the text on screen.
- `cin >> n;` → waits for the user to type a number and stores it in `n`.

```
C/C++
    Student *s = new Student[n];   // dynamic memory allocation
```

- `new Student[n]` → asks the computer for memory for `n` students **while the program is running** (this is *dynamic memory allocation*).
- `Student *s` → `s` is a **pointer**: a variable that remembers the address where that memory starts.
- After this line you can use `s[0]`, `s[1]`, ... `s[n-1]` just like an array of students.

```
C/C++
    for (int i = 0; i < n; i++) {
        cout << "\nStudent " << i + 1 << endl;
        cout << "ID: ";
        cin >> s[i].id;
        cout << "Name (one word): ";
        cin >> s[i].name;
        cout << "CGPA: ";
        cin >> s[i].cgpa;
    }
```

This loop repeats `n` times, once per student. `i` starts at 0 (`int i = 0`), the loop runs while `i < n`, and `i++` adds 1 each time.

- `cout << "\nStudent " << i + 1 << endl;` → prints "Student 1", "Student 2"... (`\n` = blank line first, `endl` = go to next line, `i + 1` because `i` starts at 0).
- `cin >> s[i].id;` → reads the ID into the `id` part of student number `i`. The dot `.` means "the part of".
- `cin >> s[i].name;` → reads the name (one word only, no spaces).
- `cin >> s[i].cgpa;` → reads the CGPA.

```C/C++
    int key, found = 0;
    cout << "\nEnter ID to search: ";
    cin >> key;
```

- `key` will hold the ID we want to find.
- `found = 0` is a flag (a signal): 0 means "not found yet", we will set it to 1 if we find it.
- Then we print a question and read the ID with `cin >> key;`.

```C/C++
    // Linear Search
    for (int i = 0; i < n; i++) {
        if (s[i].id == key) {
            cout << "Student found!" << endl;
            cout << "ID: " << s[i].id << ", Name: " << s[i].name
                 << ", CGPA: " << s[i].cgpa << endl;
            found = 1;
            break;
        }
    }
```

Linear Search: check students one by one.

- `for (int i = 0; i < n; i++)` → go through every student from 0 to n-1.
- `if (s[i].id == key)` → `==` means "is equal to". Does this student's ID match what we want?
- If yes: print the details, set `found = 1`, and `break;` (stop the loop, no need to check more).

```C/C++
    if (found == 0)
        cout << "Student not found." << endl;
```

If the loop finished and `found` is still 0, nobody matched, so print "not found".

```C/C++
    delete[] s;   // free memory
    return 0;
}
```

- `delete[] s;` → gives the reserved memory back to the computer (always do this after `new`).
- `return 0;` → tells the system the program finished normally.

Program 2: Sort Student IDs and Binary Search (C++)

Idea in simple words: We store IDs, sort them from smallest to largest, then find an ID using **Binary Search**: look at the middle element; if the ID you want is smaller, throw away the right half, if larger, throw away the left half; repeat. It is much faster than checking one by one, but it **only works on sorted data**.

```
C/C++
#include <iostream>
#include <algorithm>
using namespace std;
```

- `<iostream>` → for `cout` and `cin`.
- `<algorithm>` → gives the ready-made `sort()` function.
- `using namespace std;` → so we can write `cout`, `sort` without `std::`.

```
C/C++
int main() {
    int n;
    cout << "Enter number of student IDs: ";
    cin >> n;
```

Start of the program. `n` stores how many IDs. We print a question and read `n` with `cin`.

```
C/C++
    int id[100];
    cout << "Enter the IDs: ";
    for (int i = 0; i < n; i++)
        cin >> id[i];
```

- `int id[100];` → an array (a row of 100 boxes) for whole numbers. Boxes are numbered `id[0]`, `id[1]`, ...
- The `for` loop runs `n` times and `cin >> id[i];` reads one ID into box `i` each time.

```C/C++
    sort(id, id + n);   // ascending order

    cout << "Sorted IDs: ";
    for (int i = 0; i < n; i++)
        cout << id[i] << " ";
    cout << endl;
```

- `sort(id, id + n);` → sorts the first `n` boxes from smallest to largest. (`id` is the start, `id + n` is just after the end.)
- The next `for` loop prints all IDs with a space after each, so you can see the sorted order. `endl` goes to a new line.

```C/C++
    int key;
    cout << "Enter ID to search: ";
    cin >> key;
```

`key` is the ID to search. We ask the user and read it.

```C/C++
    // Binary Search
    int low = 0, high = n - 1, pos = -1;
```

- `low = 0` → left end of the part we are searching.
- `high = n - 1` → right end (last box).
- `pos = -1` → will hold the position found; -1 means "not found yet".

```C/C++
    while (low <= high) {
        int mid = (low + high) / 2;
        if (id[mid] == key) {
            pos = mid;
            break;
        } else if (id[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
```

Keep going while there is still something left to search (`low <= high`).

- `mid = (low + high) / 2;` → the middle position.
- `if (id[mid] == key)` → found it! Save `pos = mid;` and `break;` to leave the loop.
- `else if (id[mid] < key)` → middle value is smaller than what we want, so our ID must be on the right: `low = mid + 1;`.
- `else` → middle value is bigger, so it must be on the left: `high = mid - 1;`.

Example: IDs 10 20 30 40 50, key = 40. low=0, high=4, mid=2 → 30 < 40, so low=3. mid=3 → 40 = key. Found at index 3.

```
C/C++
    if (pos != -1)
        cout << "ID found at position " << pos + 1 << endl;
    else
        cout << "ID not found." << endl;

    return 0;
}
```

If `pos` is not -1, we found it, so print the position (`pos + 1` so it starts counting from 1). Otherwise print "not found".

Program 3: Bubble Sort of Students by CGPA (C++)

Idea in simple words: We store students and sort them by CGPA (smallest first) using **Bubble Sort**: repeatedly compare two neighbours and swap them if they are in the wrong order. After each full pass, the biggest value "bubbles" to the end.

```
C/C++
#include <iostream>
#include <string>
using namespace std;
```

Includes for printing/reading and for text (`string`), and `using namespace std;`.

```
C/C++
struct Student {
    int id;
    string name;
    float cgpa;
};
```

A `struct` groups the 3 pieces of one student's data: `id`, `name`, `cgpa`.

```C/C++
int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;
```

Start of the program. We ask how many students and store the number in `n`.

```C/C++
    Student s[100];
    for (int i = 0; i < n; i++) {
        cout << "\nStudent " << i + 1 << endl;
        cout << "ID: ";
        cin >> s[i].id;
        cout << "Name (one word): ";
        cin >> s[i].name;
        cout << "CGPA: ";
        cin >> s[i].cgpa;
    }
```

- `Student s[100];` → an array that can hold up to 100 students.
- The loop runs `n` times and reads `id`, `name` and `cgpa` for each student (`s[i].id` means "the id of student i").

```C/C++
    // Bubble Sort by CGPA (ascending)
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (s[j].cgpa > s[j + 1].cgpa) {
                Student temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
        }
    }
```

- Outer loop `for (int i ...)` → number of passes (each pass puts one more large value at its final place at the end).

- Inner loop `for (int j ...)` → goes through the neighbours. `n - i - 1` is used because the last `i` items are already in place.
- `if (s[j].cgpa > s[j + 1].cgpa)` → is this student's CGPA bigger than the next one? If yes they are in the wrong order.
- `Student temp = s[j];` → keep the first student safely in a temporary box.
- `s[j] = s[j + 1];` → copy the second into the first place.
- `s[j + 1] = temp;` → put the saved one into the second place. These 3 lines are a **swap**.

Example CGPAs 9.1, 7.5, 8.2: pass 1 → 7.5, 8.2, 9.1. Sorted.

```
C/C++
    cout << "\nRecords sorted by CGPA (ascending):" << endl;
    cout << "ID\tName\tCGPA" << endl;
    for (int i = 0; i < n; i++)
        cout << s[i].id << "\t" << s[i].name << "\t" << s[i].cgpa << endl;

    return 0;
}
```

Prints a heading, then a loop prints every student in the new sorted order. `\t` prints a tab space so columns line up.

Program 4: Binary Search with Recursion on Student IDs (C++)

Idea in simple words: Same goal as Program 2 (sort IDs, then Binary Search), but here the sort is written by hand (Bubble Sort) and the search calls **itself** (recursion): a function that solves a big problem by calling itself on a smaller part.

```
C/C++
#include <iostream>
using namespace std;
```

`<iostream>` for `cout` / `cin`, and `using namespace std;`.

```
C/C++
// Recursive Binary Search
int binarySearch(int a[], int low, int high, int key) {
    if (low > high)
        return -1;
    int mid = (low + high) / 2;
```

```
    if (a[mid] == key)
        return mid;
    else if (a[mid] < key)
        return binarySearch(a, mid + 1, high, key);
    else
        return binarySearch(a, low, mid - 1, key);
}
```

A **function** is a named block of code we can call. This one is called `binarySearch` and needs: the array `a[]`, the `low` and `high` ends, and the `key`. It returns an `int` (the position, or -1).

- `if (low > high) return -1;` → nothing left to search, so not found (this stops the recursion).
- `mid = (low + high) / 2;` → the middle position.
- `if (a[mid] == key) return mid;` → found, give back the position.
- `else if (a[mid] < key)` → the key is on the right, so **call itself** on the right part: `binarySearch(a, mid + 1, high, key)`.
- `else` → the key is on the left, so call itself on the left part: `binarySearch(a, low, mid - 1, key)`.

```
C/C++
int main() {
    int n;
    cout << "Enter number of student IDs: ";
    cin >> n;
```

Program starts here. We ask how many IDs and read `n`.

```
C/C++
    int id[100];
    cout << "Enter the IDs: ";
    for (int i = 0; i < n; i++)
        cin >> id[i];
```

An array of 100 boxes for IDs. The loop reads `n` IDs into it.

```
C/C++
    // Bubble Sort (ascending)
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = 0; j < n - i - 1; j++) {
            if (id[j] > id[j + 1]) {
                int temp = id[j];
                id[j] = id[j + 1];
                id[j + 1] = temp;
            }
        }
    }
```

Bubble Sort: compare neighbours and swap if the left one is bigger.

- The two loops go through the pairs repeatedly.
- `if (id[j] > id[j + 1])` → wrong order?
- The 3 lines with `temp` swap the two values (save one in `temp`, copy the other, put `temp` back).

```
C/C++
    cout << "Sorted IDs: ";
    for (int i = 0; i < n; i++)
        cout << id[i] << " ";
    cout << endl;
```

Print all sorted IDs using a loop.

```
C/C++
    int key;
    cout << "Enter ID to search: ";
    cin >> key;
```

Ask for the ID to search and read it into `key`.

```
C/C++
    int pos = binarySearch(id, 0, n - 1, key);

    if (pos != -1)
        cout << "ID found at position " << pos + 1 << endl;
    else
        cout << "ID not found." << endl;

    return 0;
```

```
}
```

- Calls our function on the whole array: from 0 to `n - 1`. The answer is stored in `pos`.
- If `pos` is not -1 we found it (print position + 1), else print "not found". `return 0;` ends the program.

Program 5: Stack using Linked List (C++)

Idea in simple words: A **stack** is like a pile of plates: you can only put a plate on top (**push**) and take the top plate (**pop**). A **linked list** is a chain of small boxes (nodes); each node stores a value and the address of the next node. We keep the top of the stack as the head of the chain. **Underflow** means trying to pop from an empty stack.

```
C/C++
#include <iostream>
using namespace std;
```

Includes `<iostream>` for printing/reading, `using namespace std;`.

```
C/C++
struct Node {
    int data;
    Node *next;
};
```

One node (one link of the chain):

- `int data;` → the value stored.
- `Node *next;` → a pointer to the next node (the address of the node below it).

```
C/C++
Node *top = NULL;
```

`top` points to the top node of the stack. `NULL` means "points to nothing", so the stack starts empty. It is written outside every function so all functions can use it.

```C/C++
void push(int x) {
    Node *newNode = new Node;
    newNode->data = x;
    newNode->next = top;
    top = newNode;
    cout << x << " pushed." << endl;
}
```

`push` adds a value on top.

- `Node *newNode = new Node;` → create a new empty node in memory.
- `newNode->data = x;` → put the value in it (`->` means "the part of the thing this pointer points to").
- `newNode->next = top;` → make the new node point to the old top.
- `top = newNode;` → now the new node is the top.
- Prints that the value was pushed.

```C/C++
void pop() {
    if (top == NULL) {
        cout << "Stack Underflow! Stack is empty." << endl;
        return;
    }
    Node *temp = top;
    cout << temp->data << " popped." << endl;
    top = top->next;
    delete temp;
}
```

`pop` removes the top value.

- `if (top == NULL)` → stack is empty → print **Stack Underflow** and `return;` (leave the function).
- `Node *temp = top;` → remember the top node.
- Print its value.
- `top = top->next;` → move the top down to the next node.
- `delete temp;` → free the removed node's memory.

```C/C++
void display() {
    if (top == NULL) {
```

```
        cout << "Stack is empty." << endl;
        return;
    }
    cout << "Stack elements (top to bottom): ";
    Node *p = top;
    while (p != NULL) {
        cout << p->data << " ";
        p = p->next;
    }
    cout << endl;
}
```

Shows all elements from top to bottom.

- If `top == NULL` print "empty" and leave.
- `Node *p = top;` → start at the top.
- `while (p != NULL)` → repeat until we fall off the end: print `p->data`, then `p = p->next;` moves to the next node.

```
C/C++
int main() {
    int choice, x;
    do {
        cout << "\n1. Push\n2. Pop\n3. Display\n4. Exit\nEnter choice: ";
        cin >> choice;
```

Starts the program. `choice` stores the menu option, `x` stores the value to push.

- `do { ... } while (choice != 4);` → repeats the menu again and again until the user types 4.
- The menu is printed and `cin >> choice;` reads the option.

```
C/C++
        if (choice == 1) {
            cout << "Enter value: ";
            cin >> x;
            push(x);
        } else if (choice == 2) {
            pop();
        } else if (choice == 3) {
            display();
        } else if (choice != 4) {
```

```
            cout << "Invalid choice." << endl;
        }
    } while (choice != 4);
    return 0;
}
```

Decides what to do:

- 1 → ask a value, read into `x`, call `push(x)`.
- 2 → call `pop()`.
- 3 → call `display()`.
- anything other than 4 (and not 1,2,3) → print "Invalid choice". Then `while (choice != 4)` repeats the menu; `return 0;` ends the program.

Program 6: Infix to Postfix using Array Stack (C++)

Idea in simple words: Infix is how we write maths normally: `A+B*C`. **Postfix** puts operators after their operands: `ABC*+` (no brackets needed). The trick: scan the expression left to right; letters go straight to the output; operators wait on a stack and are popped out when a stronger or equal operator comes. We make the stack with a simple array.

```
C/C++
#include <iostream>
#include <string>
#include <cctype>
using namespace std;
```

- `<iostream>` → printing/reading. `<string>` → text variables. `<cctype>` → gives `isalnum()` (checks letter/digit).
- `using namespace std;`

```
C/C++
char st[100];   // stack using array
int top = -1;

void push(char c) { st[++top] = c; }
char pop() { return st[top--]; }
```

- `char st[100];` → the stack: an array of 100 characters.
- `int top = -1;` → index of the top item; -1 means the stack is empty.

- `push(char c)` → `++top` moves top up by one, then stores `c` there.
- `pop()` → returns the item at `top`, then `top--` moves down by one.

```C/C++
int prec(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}
```

`prec` returns the **priority (precedence)** of an operator: `^` = 3 (strongest), `*` and `/` = 2, `+` and `-` = 1, anything else (like `(`) = 0.

```C/C++
int main() {
    string infix, postfix = "";
    cout << "Enter infix expression (no spaces, e.g. A+B*(C-D)): ";
    cin >> infix;
```

- `string infix, postfix = "";` → `infix` is what the user types; `postfix` is the answer we build, starting empty.
- We ask the user to type the expression (no spaces) and read it with `cin >> infix;`.

```C/C++
    for (int i = 0; i < infix.length(); i++) {
        char c = infix[i];
```

Go through the expression one character at a time. `infix[i]` is character number `i`, and we call it `c`.

```C/C++
        if (isalnum(c)) {                // operand
            postfix += c;
        } else if (c == '(') {
            push(c);
```

- If `c` is a letter or digit (an operand): add it directly to the answer (`postfix += c;`).
- If `c` is `(`: push it on the stack.

```C/C++
        } else if (c == ')') {
            while (top != -1 && st[top] != '(')
                postfix += pop();
            pop();                          // remove '('
```

If `c` is `)`: pop everything from the stack into the answer until we meet `(`. Then `pop();` once more removes the `(` itself (it is not added to the answer).

```C/C++
        } else {                            // operator
            while (top != -1 && st[top] != '(' &&
                   (prec(st[top]) > prec(c) ||
                   (prec(st[top]) == prec(c) && c != '^')))
                postfix += pop();
            push(c);
        }
    }
```

Otherwise `c` is an operator (`+ - * / ^`).

- The `while` pops operators from the stack into the answer **while** the stack is not empty, the top is not `(`, and the top has higher priority than `c` (or equal priority, except for `^`, which is calculated right-to-left).
- After popping, `push(c);` puts the new operator on the stack.

```C/C++
    while (top != -1)
        postfix += pop();

    cout << "Postfix expression: " << postfix << endl;
    return 0;
}
```

After the loop, pop all remaining operators into the answer. Then print the result.

Example `A+B*C`: A→output `A`; `+`→stack; B→`AB`; `*` has higher priority than `+`, so it just goes on the stack; C→`ABC`; end → pop `*` then `+` → *ABC*+*.

Program 7: Infix to Prefix using Array Stack (C++)

Idea in simple words: Prefix puts the operator before its operands: `A+B` becomes `+AB`. Easy method: (1) reverse the expression, (2) swap the brackets, (3) convert to postfix with the stack method, (4) reverse the answer.

```
C/C++
#include <iostream>
#include <string>
#include <algorithm>
#include <cctype>
using namespace std;
```

Includes: `<iostream>` for printing/reading, `<string>` for text, `<algorithm>` for `reverse()`, `<cctype>` for `isalnum()`.

```
C/C++
char st[100];   // stack using array
int top = -1;

void push(char c) { st[++top] = c; }
char pop() { return st[top--]; }
```

The array stack: `st` holds characters, `top = -1` means empty. `push` puts a character on top; `pop` takes the top character.

```
C/C++
int prec(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}
```

Priority of operators: `^` is 3, `* /` are 2, `+ -` are 1, others 0.

```
C/C++
int main() {
    string infix, postfix = "";
```

```
    cout << "Enter infix expression (no spaces, e.g. A+B*(C-D)): ";
    cin >> infix;
```

`infix` = what the user types, `postfix` = the answer built here. We ask and read the expression with `cin >> infix;`.

C/C++
```
    // Step 1: reverse the expression
    reverse(infix.begin(), infix.end());
```

`reverse(infix.begin(), infix.end());` flips the whole string: `A+B` becomes `B+A`.

C/C++
```
    // Step 2: swap '(' and ')'
    for (int i = 0; i < infix.length(); i++) {
        if (infix[i] == '(') infix[i] = ')';
        else if (infix[i] == ')') infix[i] = '(';
    }
```

After reversing, every `(` is facing the wrong way. The loop goes through each character: if it is `(` change it to `)`, and if it is `)` change it to `(`.

C/C++
```
    // Step 3: convert to postfix
    for (int i = 0; i < infix.length(); i++) {
        char c = infix[i];

        if (isalnum(c)) {
            postfix += c;
        } else if (c == '(') {
            push(c);
        } else if (c == ')') {
            while (top != -1 && st[top] != '(')
                postfix += pop();
            pop();
        } else {
            while (top != -1 && st[top] != '(' &&
                   (prec(st[top]) > prec(c) ||
                   (prec(st[top]) == prec(c) && c == '^')))
                postfix += pop();
            push(c);
```

```
        }
    }
    while (top != -1)
        postfix += pop();
```

Normal infix-to-postfix on the reversed string:

- Letter/digit → straight to the answer.
- `(` → push. `)` → pop to the answer until `(`, then remove `(`.
- Operator → pop from the stack while the top has **higher** priority (or equal priority only for `^`), then push the operator. (For prefix, equal priority does *not* pop, which keeps `A+B+C` correct.)
- At the end `while (top != -1) postfix += pop();` empties the stack.

```
C/C++
    // Step 4: reverse the result to get prefix
    reverse(postfix.begin(), postfix.end());

    cout << "Prefix expression: " << postfix << endl;
    return 0;
}
```

Reverse the result again to get the prefix expression and print it.

Example `A+B`: reversed `B+A` → postfix `BA+` → reversed **+AB**.

Program 8: Circular Queue using Array of Size 5 (C++)

Idea in simple words: A **queue** is like a line at a ticket counter: add at the back (**enqueue**), remove from the front (**dequeue**). In a **circular** queue, when the end of the array is reached, the back wraps around to position 0 so empty space at the front can be reused.

```
C/C++
#include <iostream>
using namespace std;

#define SIZE 5
```

- `<iostream>` for printing/reading.
- `#define SIZE 5` → wherever we write `SIZE`, it means 5 (the queue's capacity).

```C/C++
int q[SIZE];
int front = -1, rear = -1;
```

- `q[SIZE]` → the array that holds the queue.
- `front` → position of the first item; `rear` → position of the last item.
- `-1` for both means the queue is empty.

```C/C++
void enqueue(int x) {
    if ((rear + 1) % SIZE == front) {
        cout << "Queue is FULL!" << endl;
        return;
    }
    if (front == -1)
        front = 0;
    rear = (rear + 1) % SIZE;
    q[rear] = x;
    cout << x << " enqueued." << endl;
}
```

Adds `x` at the back.

- `(rear + 1) % SIZE == front` → if moving rear forward by one (wrapping around with `%`) would hit front, the queue is **FULL**: print message and `return;`.
- `if (front == -1) front = 0;` → first item ever added, so front becomes 0.
- `rear = (rear + 1) % SIZE;` → move rear forward; `%` makes 5 wrap back to 0.
- `q[rear] = x;` → store the value.

```C/C++
void dequeue() {
    if (front == -1) {
        cout << "Queue is EMPTY!" << endl;
        return;
    }
    cout << q[front] << " dequeued." << endl;
    if (front == rear)
        front = rear = -1;          // queue became empty
    else
        front = (front + 1) % SIZE;
}
```

Removes from the front.

- `front == -1` → queue is **EMPTY**: print message and `return;`.
- Print `q[front]` (the item being removed).
- `if (front == rear)` → that was the only item, so reset both to -1 (empty again).
- `else front = (front + 1) % SIZE;` → move front forward with wrap-around.

```C/C++
void display() {
    if (front == -1) {
        cout << "Queue is EMPTY!" << endl;
        return;
    }
    cout << "Queue elements: ";
    int i = front;
    while (true) {
        cout << q[i] << " ";
        if (i == rear) break;
        i = (i + 1) % SIZE;
    }
    cout << endl;
}
```

Prints items from front to rear.

- If empty, print message.
- `int i = front;` → start at the front.
- `while (true)` → repeat forever until we `break`: print `q[i]`; if `i == rear` we printed the last one, so `break;`; otherwise `i = (i + 1) % SIZE;` moves to the next (wrapping).

```C/C++
int main() {
    int choice, x;
    do {
        cout << "\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\nEnter choice: ";
        cin >> choice;
```

Program start. `choice` is the menu option, `x` is the value to add.

- `do { ... } while (choice != 4);` repeats the menu until the user types 4.

```C/C++
        if (choice == 1) {
            cout << "Enter value: ";
            cin >> x;
            enqueue(x);
        } else if (choice == 2) {
            dequeue();
        } else if (choice == 3) {
            display();
        } else if (choice != 4) {
            cout << "Invalid choice." << endl;
        }
    } while (choice != 4);
    return 0;
}
```

- 1 → ask a value and call `enqueue(x)`.
- 2 → call `dequeue()`.
- 3 → call `display()`.
- other than 4 → "Invalid choice". `return 0;` ends the program.

PART B: Python

Program 9: BST: Create, Traversals and Duplicate Handling (Python)

Idea in simple words: A **BST (Binary Search Tree)** is a tree where each box (node) has at most 2 children. Rule: everything in the **left** part is **smaller** than the node, everything in the **right** part is **bigger**. We insert numbers one by one, and then print the tree in 3 different orders (**traversals**): inorder (left, node, right), preorder (node, left, right) and postorder (left, right, node).

```Python
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

A **class** is a blueprint for making objects. Here `Node` is the blueprint for one box of the tree.

- `class Node:` → start of the blueprint. The indented lines belong to it.
- `def __init__(self, key):` → the special function that runs when a new Node is created. `self` means "this node itself"; `key` is the value we give it.
- `self.key = key` → store the value inside the node.
- `self.left = None` and `self.right = None` → the node starts with no left child and no right child. `None` means "nothing".

```python
# Rule for duplicates: a duplicate value is NOT inserted (it is ignored)
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    else:
        print("Duplicate", key, "ignored")
    return root
```

The **rule for duplicates** we chose: if the number is already in the tree, we do **not** add it again; we just print a message.

- `insert` works the same as described before: empty place → make a node; smaller → go left; bigger → go right.
- `else:` → the number is neither smaller nor bigger, so it is equal (a duplicate). `print("Duplicate", key, "ignored")` shows a message.
- `return root` gives back the tree.

```python
def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)
```

Inorder = left, node, right.

- `def inorder(root):` → function to print the tree.
- `if root:` → only continue if the node exists (not `None`).
- `inorder(root.left)` → first visit everything on the left,
- `print(root.key, end=" ")` → then print this node's value (`end=" "` keeps the output on one line with a space),

- `inorder(root.right)` → then everything on the right. For a BST this prints values in **sorted order**.

```python
def preorder(root):
    if root:
        print(root.key, end=" ")
        preorder(root.left)
        preorder(root.right)
```

Preorder = node first, then left, then right. Same as inorder, but the `print` line comes **before** the two recursive calls.

```python
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.key, end=" ")
```

Postorder = left, right, then the node. The `print` line comes **after** both recursive calls.

```python
values = list(map(int, input("Enter values separated by space: ").split()))
```

The main program starts here.

- `input("...")` shows the message and waits for the user to type; it gives back the typed text, e.g. `"50 30 70"`.
- `.split()` cuts the text at spaces into a list of pieces: `["50", "30", "70"]`.
- `map(int, ...)` converts every piece from text into a number.
- `list(...)` makes it a proper list `[50, 30, 70]`, stored in `values`.

```python
root = None
for v in values:
    root = insert(root, v)
```

- `root = None` → the tree is empty at the start.
- `for v in values:` → take each number from the list one by one, calling it `v`.

- `root = insert(root, v)` → insert it and save the updated tree.

```python
print("Inorder   : ", end="")
inorder(root)
print("\nPreorder  : ", end="")
preorder(root)
print("\nPostorder : ", end="")
postorder(root)
print()
```

- `print("Inorder   : ", end="")` → prints the label and stays on the same line.
- `inorder(root)` → prints the values.
- The same is done for preorder and postorder. In the labels `\n` means "start a new line first".
- The last `print()` just moves to a new line.

Program 10: BST: Insertion and Search (Python)

Idea in simple words: A **BST (Binary Search Tree)** is a tree where each box (node) has at most 2 children. Rule: everything in the **left** part is **smaller** than the node, everything in the **right** part is **bigger**. We insert numbers and then **search** for a key: at each node, if the key is smaller go left, if larger go right, until we find it or run out of tree.

```python
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

A **class** is a blueprint for making objects. Here `Node` is the blueprint for one box of the tree.

- `class Node:` → start of the blueprint. The indented lines belong to it.
- `def __init__(self, key):` → the special function that runs when a new Node is created. `self` means "this node itself"; `key` is the value we give it.
- `self.key = key` → store the value inside the node.
- `self.left = None` and `self.right = None` → the node starts with no left child and no right child. `None` means "nothing".

```python
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root
```

`insert` puts a new value in the correct place and returns the (updated) tree. It uses **recursion** (the function calls itself on a smaller part).

- `def insert(root, key):` → defines a function. `root` is the tree (or sub-tree) we look at, `key` is the value to add.
- `if root is None:` → this place is empty, so
- `return Node(key)` → create a new node with the value here and give it back.
- `if key < root.key:` → smaller → belongs on the left, so `root.left = insert(root.left, key)` calls insert again on the left side and saves the result.
- `elif key > root.key:` → bigger → same idea on the right side.
- `return root` → give back the tree.

```python
def search(root, key):
    if root is None:
        return False
    if root.key == key:
        return True
    if key < root.key:
        return search(root.left, key)
    return search(root.right, key)
```

`search` returns `True` if the key is in the tree, else `False`.

- `if root is None: return False` → we reached an empty place, so the key is not there.
- `if root.key == key: return True` → this node has the key. (`==` means "is equal to".)
- `if key < root.key: return search(root.left, key)` → smaller → look in the left part.
- `return search(root.right, key)` → otherwise look in the right part.

```python
values = list(map(int, input("Enter values to insert (space separated):
").split()))

root = None
for v in values:
    root = insert(root, v)
```

- `input(...)` reads what the user types as text, `.split()` cuts it at spaces into a list of text pieces, `map(int, ...)` turns each piece into a number, `list(...)` makes a list.
- `values` now holds the numbers to insert.
- Then `root = None` (the tree starts empty) and `for v in values: root = insert(root, v)` inserts each number one by one.

```python
key = int(input("Enter key to search: "))
```

Reads the key to search. `input(...)` gives text and `int(...)` converts it to a number.

```python
if search(root, key):
    print("Key", key, "is PRESENT in the BST")
else:
    print("Key", key, "is NOT present in the BST")
```

Call `search`. If it returns `True` print "PRESENT", otherwise (`else:`) print "NOT present".

Program 11: BST: Deletion (leaf, one child, two children) (Python)

Idea in simple words: A **BST (Binary Search Tree)** is a tree where each box (node) has at most 2 children. Rule: everything in the **left** part is **smaller** than the node, everything in the **right** part is **bigger**. Deleting a key has 3 cases: **(1) leaf** (no children) → just remove it; **(2) one child** → replace the node by its child; **(3) two children** → replace its value with the **inorder successor** (the smallest value in its right part), then delete that successor.

```python
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
```

```
        self.right = None
```

A **class** is a blueprint for making objects. Here `Node` is the blueprint for one box of the tree.

- `class Node:` → start of the blueprint. The indented lines belong to it.
- `def __init__(self, key):` → the special function that runs when a new Node is created. `self` means "this node itself"; `key` is the value we give it.
- `self.key = key` → store the value inside the node.
- `self.left = None` and `self.right = None` → the node starts with no left child and no right child. `None` means "nothing".

Python

```python
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root
```

`insert` puts a new value in the correct place and returns the (updated) tree. It uses **recursion** (the function calls itself on a smaller part).

- `def insert(root, key):` → defines a function. `root` is the tree (or sub-tree) we look at, `key` is the value to add.
- `if root is None:` → this place is empty, so
- `return Node(key)` → create a new node with the value here and give it back.
- `if key < root.key:` → smaller → belongs on the left, so `root.left = insert(root.left, key)` calls insert again on the left side and saves the result.
- `elif key > root.key:` → bigger → same idea on the right side.
- `return root` → give back the tree.

Python

```python
def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)
```

- `def inorder(root):` → function to print the tree.
- `if root:` → only continue if the node exists (not `None`).
- `inorder(root.left)` → first visit everything on the left,
- `print(root.key, end=" ")` → then print this node's value (`end=" "` keeps the output on one line with a space),
- `inorder(root.right)` → then everything on the right. For a BST this prints values in **sorted order**.

```python
def min_node(node):
    while node.left:
        node = node.left
    return node
```

Finds the smallest node in a tree: keep going left.

- `while node.left:` → repeat as long as the node has a left child.
- `node = node.left` → move to it.
- `return node` → the leftmost node is the smallest.

```python
def delete(root, key):
    if root is None:
        print("Key not found")
        return None
```

`delete` removes a key and returns the updated tree.

- `if root is None:` → empty place, the key is not in the tree, print "Key not found" and `return None`.

```python
    if key < root.key:
        root.left = delete(root.left, key)
    elif key > root.key:
        root.right = delete(root.right, key)
    else:
```

Finding the node:

- key smaller → look in the left part: `root.left = delete(root.left, key)`.

- `elif key > root.key:` → key bigger → look in the right part.
- `else:` → we found the node to delete, and the 3 cases follow.

```python
        # Case 1: leaf node
        if root.left is None and root.right is None:
            return None
```

Case 1: leaf. No left and no right child → `return None` (the node disappears).

```python
        # Case 2: one child
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
```

Case 2: one child.

- If there is no left child, return the right child (it takes the node's place).
- If there is no right child, return the left child.

```python
        # Case 3: two children -> use inorder successor
        succ = min_node(root.right)
        root.key = succ.key
        root.right = delete(root.right, succ.key)
    return root
```

Case 3: two children.

- `succ = min_node(root.right)` → find the smallest value in the right part (inorder successor).
- `root.key = succ.key` → copy that value into this node.
- `root.right = delete(root.right, succ.key)` → remove the original successor node from the right part.
- `return root` → give back the tree.

```python
values = list(map(int, input("Enter values to insert (space separated):
").split()))

root = None
for v in values:
    root = insert(root, v)

print("Inorder before deletion: ", end="")
inorder(root)
print()
```

Read the numbers: `input` → text, `.split()` → list of pieces, `map(int, ...)` → numbers, `list(...)` → list. Then `root = None` starts an empty tree and the `for` loop inserts each number.

```python
key = int(input("Enter key to delete: "))
root = delete(root, key)

print("Inorder after deletion : ", end="")
inorder(root)
print()
```

- Reads the key to delete as a number.
- `root = delete(root, key)` → deletes it and saves the updated tree.
- Before and after, `inorder(root)` is printed so you can compare.

Program 12: BST: Depth and Leaf Nodes (Python)

Idea in simple words: A **BST (Binary Search Tree)** is a tree where each box (node) has at most 2 children. Rule: everything in the **left** part is **smaller** than the node, everything in the **right** part is **bigger**. **Depth** = how many levels the tree has. **Leaf nodes** = nodes with no children.

```python
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

A **class** is a blueprint for making objects. Here `Node` is the blueprint for one box of the tree.

- `class Node:` → start of the blueprint. The indented lines belong to it.
- `def __init__(self, key):` → the special function that runs when a new Node is created. `self` means "this node itself"; `key` is the value we give it.
- `self.key = key` → store the value inside the node.
- `self.left = None` and `self.right = None` → the node starts with no left child and no right child. `None` means "nothing".

```python
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root
```

`insert` puts a new value in the correct place and returns the (updated) tree. It uses **recursion** (the function calls itself on a smaller part).

- `def insert(root, key):` → defines a function. `root` is the tree (or sub-tree) we look at, `key` is the value to add.
- `if root is None:` → this place is empty, so
- `return Node(key)` → create a new node with the value here and give it back.
- `if key < root.key:` → smaller → belongs on the left, so `root.left = insert(root.left, key)` calls insert again on the left side and saves the result.
- `elif key > root.key:` → bigger → same idea on the right side.
- `return root` → give back the tree.

```python
# depth = number of levels (empty tree = 0)
def depth(root):
    if root is None:
        return 0
    return 1 + max(depth(root.left), depth(root.right))
```

`depth` counts the levels.

- `if root is None: return 0` → an empty place has 0 levels.

- `return 1 + max(depth(root.left), depth(root.right))` → this node is 1 level, plus the deeper of its left and right parts. `max(a, b)` gives the bigger number.

```python
def leaves(root):
    if root is None:
        return
    if root.left is None and root.right is None:
        print(root.key, end=" ")
    leaves(root.left)
    leaves(root.right)
```

Prints leaf nodes.

- `if root is None: return` → nothing to do for an empty place (`return` just leaves the function).
- `if root.left is None and root.right is None:` → no left child **and** no right child means it is a leaf, so print its key.
- Then call `leaves` on the left part and the right part so every node is checked.

```python
values = list(map(int, input("Enter values to insert (space separated):
").split()))

root = None
for v in values:
    root = insert(root, v)
```

Reads numbers (`input` → `.split()` → `map(int,...)` → `list`). Then `root = None` and a `for` loop insert each number into the tree.

```python
print("Depth of BST (levels):", depth(root))
print("Leaf nodes: ", end="")
leaves(root)
print()
```

Prints the depth by calling `depth(root)` and then prints the leaf nodes by calling `leaves(root)` (the `end=" "` inside keeps them on one line).

Program 13: Kruskal's Algorithm with Adjacency List (Python)

Idea in simple words: A graph is places (vertices) joined by roads (edges) with distances (weights). A **Minimum Spanning Tree (MST)** connects all places using the smallest total distance and no loops. **Kruskal's idea:** sort all roads from shortest to longest, and take a road only if it does **not** make a loop. An **adjacency list** stores, for each place, the list of its neighbours.

```python
# Campus graph using adjacency list: building -> [(neighbour, distance), ...]
graph = {
    "Gate":      [("Admin", 4), ("Canteen", 8)],
    "Admin":     [("Gate", 4), ("Library", 3), ("CS Dept", 6)],
    "Library":   [("Admin", 3), ("CS Dept", 2), ("Hostel", 7)],
    "CS Dept":   [("Admin", 6), ("Library", 2), ("Canteen", 5)],
    "Canteen":   [("Gate", 8), ("CS Dept", 5), ("Hostel", 4)],
    "Hostel":    [("Library", 7), ("Canteen", 4)],
}
```

A **dictionary** `{ key: value }` stores pairs. Here each key is a place and each value is a **list** `[ ... ]` of tuples `(neighbour, distance)`. Example: `"Gate": [("Admin", 4), ("Canteen", 8)]` means Gate–Admin is 4 and Gate–Canteen is 8. This is the adjacency list.

```python
# Step 1: collect every edge once (weight, u, v)
edges = []
for u in graph:
    for v, w in graph[u]:
        if (w, v, u) not in edges:
            edges.append((w, u, v))
```

Collect every road once.

- `edges = []` → an empty list.
- `for u in graph:` → go through each place (`u`).
- `for v, w in graph[u]:` → go through its neighbours: `v` is the neighbour, `w` the distance.
- Each road appears twice (Gate→Admin and Admin→Gate). `if (w, v, u) not in edges:` checks that the reverse is not already added.
- `edges.append((w, u, v))` → add the road as (distance, place1, place2). The distance is first so sorting works by distance.

```python
# Step 2: sort edges by weight
edges.sort()
```

`edges.sort()` sorts the list from the shortest road to the longest.

```python
# Union-Find to detect cycles
parent = {node: node for node in graph}

def find(x):
    while parent[x] != x:
        x = parent[x]
    return x
```

Tool to detect loops: every place starts as its own group.

- `parent = {node: node for node in graph}` → for each place, store itself as its group leader.
- `def find(x):` → follows the leaders until it reaches the top leader of the group of `x`.
- `while parent[x] != x:` → keep moving up while `x` is not its own leader; `x = parent[x]` moves up.
- `return x` → the group leader.

```python
# Step 3: Kruskal's algorithm
mst = []
total = 0
for w, u, v in edges:
    root_u = find(u)
    root_v = find(v)
    if root_u != root_v:          # no cycle
        parent[root_u] = root_v
        mst.append((u, v, w))
        total += w
```

Kruskal's main loop.

- `mst = []` (chosen roads) and `total = 0`.
- `for w, u, v in edges:` → take roads shortest first.
- `root_u = find(u)` and `root_v = find(v)` → group leaders of both ends.

- `if root_u != root_v:` → they are in different groups, so joining them makes no loop.
- `parent[root_u] = root_v` → merge the two groups.
- `mst.append((u, v, w))` → keep the road, and `total += w` adds its distance.

```python
print("Edges in Minimum Spanning Tree:")
for u, v, w in mst:
    print(u, "-", v, ":", w)
print("Total weight of MST =", total)
```

Prints every chosen road with its distance, then the total weight.

Program 14: Prim's Algorithm with Adjacency Matrix (Python)

Idea in simple words: Same goal as Kruskal (Minimum Spanning Tree) but a different method: **start at one place and keep growing the tree**: always add the cheapest road that connects a new place. An **adjacency matrix** is a table: row i, column j gives the distance from place i to place j (0 = no road).

```python
# Campus graph using adjacency matrix (0 = no direct path)
names = ["Gate", "Admin", "Library", "CS Dept", "Canteen", "Hostel"]

matrix = [
    #  G  A  L  C  Ca H
    [0, 4, 0, 0, 8, 0],   # Gate
    [4, 0, 3, 6, 0, 0],   # Admin
    [0, 3, 0, 2, 0, 7],   # Library
    [0, 6, 2, 0, 5, 0],   # CS Dept
    [8, 0, 0, 5, 0, 4],   # Canteen
    [0, 0, 7, 0, 4, 0],   # Hostel
]
```

- `names` is a list of the place names. Index 0 = Gate, 1 = Admin, and so on.
- `matrix` is a list of lists (a table). `matrix[0][1]` is 4: the distance from Gate (row 0) to Admin (column 1). `0` means no direct road.

```python
n = len(names)
INF = 999999
```

```
selected = [False] * n      # vertices already in MST
key = [INF] * n             # smallest edge weight to reach each vertex
parent = [-1] * n
```

- `n = len(names)` → number of places (`len` counts the items).
- `INF = 999999` → a very big number meaning "not reachable yet".
- `selected = [False] * n` → a list of `n` False values: no place is in the tree yet.
- `key = [INF] * n` → cheapest road to reach each place so far (all huge at first).
- `parent = [-1] * n` → which place each place is connected from.

Python
```
key[0] = 0                  # start from vertex 0
total = 0
```

Start from place 0 (cost 0). `total = 0` will add up the road distances.

Python
```
for _ in range(n):
    # pick the unselected vertex with the smallest key
    u = -1
    for i in range(n):
        if not selected[i] and (u == -1 or key[i] < key[u]):
            u = i
```

Repeat `n` times (once per place; `_` is just a name for "I don't need this number").

- `u = -1` → no place chosen yet.
- `for i in range(n):` → go through all places.
- `if not selected[i] and (u == -1 or key[i] < key[u]):` → consider only places not yet in the tree, and pick the one with the smallest `key`.
- `u = i` → remember it.

Python
```
    selected[u] = True
    total += key[u]
```

Add the chosen place `u` to the tree (`selected[u] = True`) and add its cost to `total`.

```python
    # update keys of neighbours
    for v in range(n):
        if matrix[u][v] != 0 and not selected[v] and matrix[u][v] < key[v]:
            key[v] = matrix[u][v]
            parent[v] = u
```

Look at all places `v`: if there is a road from `u` to `v` (`matrix[u][v] != 0`), `v` is not yet in the tree, and this road is cheaper than the best known one, then update `key[v]` and remember `parent[v] = u`.

```python
print("Edges in Minimum Spanning Tree (Prim's):")
for v in range(1, n):
    print(names[parent[v]], "-", names[v], ":", matrix[parent[v]][v])
print("Total weight of MST =", total)
```

`for v in range(1, n):` goes through places 1 to n-1 and prints the road `parent[v]` – `v` with its distance. Finally prints the total weight.

Program 15: Heap Sort (Python)

Idea in simple words: A **max-heap** is a tree stored in a list where every parent is bigger than its children, so the biggest value is always at the front (index 0). Heap Sort: (1) arrange the list as a max-heap, (2) swap the biggest to the end, shrink the heap, fix it, repeat. In the list, the children of index `i` are at `2*i+1` (left) and `2*i+2` (right). It is in-place (no extra list) and takes O(n log n) time.

```python
# Heap Sort using a MAX-HEAP (in-place, time O(n log n), extra space O(1))
```

A comment explaining the idea. Lines starting with `#` are ignored by Python.

```python
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
```

`heapify` fixes the heap for the part starting at index `i` (only the first `n` items count).

- `largest = i` → assume the parent is the biggest.
- `left = 2 * i + 1` and `right = 2 * i + 2` → positions of its two children.

```python
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
```

- If the left child exists (`left < n`) and is bigger than the biggest so far, `largest = left`.
- Same check for the right child.
- `if largest != i:` → a child is bigger than the parent, so swap them: `arr[i], arr[largest] = arr[largest], arr[i]` (Python swaps two values in one line).
- `heapify(arr, n, largest)` → the moved value may break the rule lower down, so fix there too (recursion).

```python
def heap_sort(arr):
    n = len(arr)
```

- `n = len(arr)` → number of items.

```python
    # Step 1: build max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
```

Build the max-heap: `range(n // 2 - 1, -1, -1)` counts down from the last parent to 0 (`//` is whole-number division). Calling `heapify` on each makes the whole list a max-heap.

```python
    # Step 2: move the largest element to the end, then fix the heap
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
```

```
        heapify(arr, i, 0)
```

Sorting:

- `for i in range(n - 1, 0, -1):` → `i` goes from the last index down to 1.
- `arr[0], arr[i] = arr[i], arr[0]` → swap the biggest (front) with position `i`, so the biggest goes to its final place.
- `heapify(arr, i, 0)` → heap size is now `i`; fix the new front.

```
Python
arr = list(map(int, input("Enter numbers separated by space: ").split()))
print("Original array:", arr)
heap_sort(arr)
print("Sorted array  :", arr)
```

- Read numbers: `input` → `.split()` → `map(int, ...)` → `list`.
- Print the original, call `heap_sort(arr)` (it sorts the same list in place), and print the sorted list.

Program 16: Merge Sort for Delivery Orders (Python)

Idea in simple words: Merge Sort splits the list in half again and again until each piece has 1 item (already sorted), then **merges** the sorted pieces back together by repeatedly taking the smaller front item. Here each order is a pair (name, minutes) and we sort by minutes, so quicker deliveries come first.

```
Python
def merge_sort(orders):
    if len(orders) <= 1:
        return orders
```

- `def merge_sort(orders):` → function that sorts a list.
- `if len(orders) <= 1: return orders` → a list of 0 or 1 items is already sorted (this stops the recursion).

```
Python
    mid = len(orders) // 2
    left = merge_sort(orders[:mid])
```

```
    right = merge_sort(orders[mid:])
```

- `mid` → the middle position (`//` = whole-number division).
- `orders[:mid]` → the first half; `orders[mid:]` → the second half.
- Each half is sorted by calling `merge_sort` on it (recursion).

Python
```
    # merge two sorted halves (sort by delivery time)
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i][1] <= right[j][1]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
```

Merging two sorted lists.

- `result = []` → the merged list; `i = j = 0` → two pointers, one for `left`, one for `right`.
- `while i < len(left) and j < len(right):` → while both still have items.
- `left[i][1]` → the minutes of the order (`[0]` is the name, `[1]` the time).
- If the left one is smaller or equal, add it to `result` and move `i`; else add the right one and move `j`. (`i += 1` means add 1 to `i`.)

Python
```
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

One side may have items left over (already sorted). `extend` adds them all to the end. `return result` gives back the merged list.

Python
```
n = int(input("Enter number of orders: "))
orders = []
for i in range(n):
```

```
    t = int(input("Delivery time (minutes) for Order " + str(i + 1) + ": "))
    orders.append(("Order " + str(i + 1), t))
```

Main program:

- Ask how many orders (`int(...)` converts the typed text to a number).
- `orders = []` → empty list.
- `for i in range(n):` → repeat `n` times. Read the time for each order and add `("Order 1", time)` etc. using `orders.append(...)`. (`str(i + 1)` turns a number into text so it can be joined with `+`.)

Python
```
sorted_orders = merge_sort(orders)

print("\nOrders sorted by delivery time (quickest first):")
for name, t in sorted_orders:
    print(name, "->", t, "minutes")
```

Calls `merge_sort` to get the sorted list, prints a heading, then `for name, t in sorted_orders:` takes each pair (name and time) and prints it.

Program 17: Fractional Knapsack (Python)

Idea in simple words: A truck can carry limited weight. Each parcel has a weight and a profit and we may take **part** of a parcel. **Greedy idea:** the best parcel is the one with the highest profit per kg (profit ÷ weight). Take the best ones first; when the next parcel does not fit fully, take just the part that fits.

Python
```
# Fractional Knapsack - greedy method (highest profit/weight first)

n = int(input("Enter number of parcels: "))
parcels = []
for i in range(n):
    w = float(input("Weight of parcel " + str(i + 1) + ": "))
    p = float(input("Profit of parcel " + str(i + 1) + ": "))
    parcels.append((i + 1, w, p))
```

Reading parcels:

- `n` = number of parcels (`int(input(...))`: read text, convert to whole number).
- `parcels = []` → empty list.
- `for i in range(n):` → repeat `n` times, reading weight `w` and profit `p` (`float` = decimal number).
- `parcels.append((i + 1, w, p))` → store (parcel number, weight, profit) as a tuple.

```python
capacity = float(input("Enter truck capacity: "))
```

Reads the truck capacity.

```python
# sort by profit/weight ratio (highest first)
parcels.sort(key=lambda x: x[2] / x[1], reverse=True)
```

Sort the parcels so the highest profit-per-weight comes first.

- `key=lambda x: x[2] / x[1]` → a `lambda` is a tiny one-line function: for each parcel `x`, the sorting value is profit (`x[2]`) divided by weight (`x[1]`).
- `reverse=True` → biggest first.

```python
total_profit = 0
remaining = capacity

print("\nParcels loaded:")
```

- `total_profit` → profit collected so far.
- `remaining = capacity` → space left in the truck.
- Prints a heading.

```python
for num, w, p in parcels:
    if remaining == 0:
        break
```

Go through the sorted parcels (unpacking each tuple into `num`, `w`, `p`).

- `if remaining == 0: break` → truck is full, stop.

```python
    if w <= remaining:                  # take whole parcel
        print("Parcel", num, ": full  (weight", w, ", profit", p, ")")
        total_profit += p
        remaining -= w
```

If the whole parcel fits (`w <= remaining`): print it, add its full profit, and reduce the remaining space by its weight (`-=` means subtract and store).

```python
    else:                               # take a fraction
        fraction = remaining / w
        print("Parcel", num, ": partial", round(fraction * 100, 2), "% (profit",
round(p * fraction, 2), ")")
        total_profit += p * fraction
        remaining = 0
```

Otherwise only a part fits:

- `fraction = remaining / w` → the part we can take (e.g. 0.67).
- Print the percentage taken and the profit from it (`round(x, 2)` keeps 2 decimals).
- Add `p * fraction` to the profit and set `remaining = 0` (truck is full).

```python
print("\nMaximum profit =", round(total_profit, 2))
```

Prints the final total profit.

Program 18: Naive String Matching (Python)

Idea in simple words: We want every position where the pattern appears inside the text. **Naive method:** try the pattern at every starting position and compare letter by letter (here as a slice).

```python
# Naive String Matching Algorithm

text = input("Enter text   : ")
pattern = input("Enter pattern: ")
```

- `text = input("Enter text  : ")` → reads the text; `pattern` is read the same way.
- Text is just a sequence of characters; `text[0]` is the first character.

```python
n = len(text)
m = len(pattern)

found = []
```

- `n` = length of the text, `m` = length of the pattern (`len` counts characters).
- `found = []` → an empty list to collect the starting positions.

```python
for i in range(n - m + 1):
    if text[i:i + m] == pattern:      # compare substring with pattern
        found.append(i)
```

- `range(n - m + 1)` → possible starting positions `0` to `n - m` (the pattern must fit).
- `text[i:i + m]` → a **slice**: the part of text from position `i` up to (not including) `i + m`, exactly `m` characters.
- `if ... == pattern:` → if it equals the pattern, `found.append(i)` stores the position.

```python
if len(found) == 0:
    print("Pattern not found")
else:
    print("Pattern found at indices:", found)
```

If the list is empty print "not found", otherwise print all positions found.

Program 19: BFS Shortest Path in a Maze (Python)

Idea in simple words: The maze is a table of cells: `0` = free, `1` = obstacle. **BFS (Breadth-First Search)** explores cells in rings: first all cells 1 step away, then 2 steps, and so on, so the first time it reaches the goal is a shortest path.

```python
from collections import deque
```

`import`-type line: `from collections import deque` brings in `deque`, a list that is fast for taking items from the front. We use it as a **queue** (first in, first out).

```python
# 0 = free cell, 1 = obstacle
maze = [
    [0, 0, 1, 0, 0],
    [1, 0, 1, 0, 1],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
]
```

`maze` is a list of rows; each row is a list of numbers. `maze[2][1]` means row 2, column 1.

```python
start = (0, 0)
goal = (4, 4)
```

`(0, 0)` is a **tuple** (row, column). Start is the top-left cell, goal is `(4, 4)` bottom-right.

```python
# BFS
queue = deque([start])
parent = {start: None}
```

- `queue = deque([start])` → the queue starts with only the start cell.
- `parent = {start: None}` → a dictionary remembering which cell we came from. The start has no parent (`None`). It also tells us which cells are already visited.

```python
while queue:
    r, c = queue.popleft()
    if (r, c) == goal:
        break
```

Repeat while the queue is not empty.

- `r, c = queue.popleft()` → take the first cell out of the queue (its row `r` and column `c`).
- `if (r, c) == goal: break` → we reached the goal, stop.

```
Python
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 5 and 0 <= nc < 5 and maze[nr][nc] == 0 and (nr, nc) not in
parent:
            parent[(nr, nc)] = (r, c)
            queue.append((nr, nc))
```

Look at the 4 neighbours: up `(-1, 0)`, down `(1, 0)`, left `(0, -1)`, right `(0, 1)`.

- `nr, nc = r + dr, c + dc` → neighbour's row and column.
- The `if` checks: inside the maze (`0 <= nr < 5 and 0 <= nc < 5`), it is free (`maze[nr][nc] == 0`), and not visited (`not in parent`).
- Then `parent[(nr, nc)] = (r, c)` records where we came from, and `queue.append((nr, nc))` adds it to the queue.

```
Python
if goal not in parent:
    print("No path exists")
else:
```

If the goal was never reached, print "No path exists".

```
Python
    path = []
    cell = goal
    while cell is not None:
        path.append(cell)
        cell = parent[cell]
    path.reverse()
```

Build the path backwards:

- Start at the goal; `while cell is not None:` add the cell to `path`, then move to its parent: `cell = parent[cell]`. The start has parent `None`, which ends the loop.

- `path.reverse()` flips the list so it goes start → goal.

```python
    print("Shortest path length:", len(path) - 1)
    print("Path:", path)
```

- `len(path) - 1` → number of steps (cells minus 1).
- Prints the length and the path.

Program 20: A* Search in a Maze (Python)

Idea in simple words: A* is like BFS but smarter: it prefers cells that look closer to the goal. For each cell: **g** = steps taken so far, **h** = estimated distance to the goal (**Manhattan distance** = |row difference| + |column difference|), **f = g + h**. It always explores the cell with the smallest f first, using a **priority queue** (`heapq`) that always gives the smallest item.

```python
import heapq
```

`import heapq` brings in tools for a priority queue (a list where the smallest item comes out first).

```python
# 0 = free cell, 1 = obstacle
maze = [
    [0, 0, 1, 0, 0],
    [1, 0, 1, 0, 1],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
]
```

The maze: `0` = free, `1` = obstacle (a list of rows).

```python
start = (0, 0)
goal = (4, 4)
```

Start cell `(0, 0)` and goal cell `(4, 4)` as (row, column) tuples.

```python
def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])   # Manhattan distance
```

The Manhattan distance: `abs(...)` gives the positive difference, so we add the row difference and column difference.

```python
# A* search
open_list = [(heuristic(start, goal), 0, start)]   # (f, g, cell)
g_cost = {start: 0}
parent = {start: None}
```

- `open_list = [(heuristic(start, goal), 0, start)]` → the priority queue holds tuples `(f, g, cell)`; it is sorted by `f` first.
- `g_cost = {start: 0}` → best known steps to reach each cell (start costs 0).
- `parent = {start: None}` → where each cell was reached from.

```python
while open_list:
    f, g, cell = heapq.heappop(open_list)
    if cell == goal:
        break
```

Repeat while there are cells to explore.

- `heapq.heappop(open_list)` → takes out the tuple with the smallest `f`; we unpack it into `f, g, cell`.
- If `cell == goal` → done, `break`.

```python
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = cell[0] + dr, cell[1] + dc
        if 0 <= nr < 5 and 0 <= nc < 5 and maze[nr][nc] == 0:
            new_g = g + 1
            if (nr, nc) not in g_cost or new_g < g_cost[(nr, nc)]:
                g_cost[(nr, nc)] = new_g
                parent[(nr, nc)] = cell
                f_new = new_g + heuristic((nr, nc), goal)
```

```
                heapq.heappush(open_list, (f_new, new_g, (nr, nc)))
```

Check the 4 neighbours (up, down, left, right): `nr, nc` is the neighbour's position.

- Must be inside the maze and free (`maze[nr][nc] == 0`).
- `new_g = g + 1` → one more step.
- If this cell is new, or we found a shorter way (`new_g < g_cost[...]`): save the cost, save the parent, compute `f_new = new_g + heuristic(...)` and add `(f_new, new_g, (nr, nc))` with `heapq.heappush`.

Python
```
if goal not in parent:
    print("No path exists")
else:
```

If the goal was never reached, print "No path exists".

Python
```
    path = []
    cell = goal
    while cell is not None:
        path.append(cell)
        cell = parent[cell]
    path.reverse()
```

Build the path backwards from goal using `parent` until `None`, then `path.reverse()` makes it start → goal.

Python
```
    print("Path length:", len(path) - 1)
    print("Path:", path)
```

Prints the number of steps (`len(path) - 1`) and the path.